

Route Change

Resolução UVa 11833 - Algoritmos em
Grafos

Grupo: Aaron Magno, Arthur Soares, Davi Dias
Prof: Ricardo Carubbi

O Contexto do Problema

Análise do sistema de transporte da Dias Transport

I^{*} ^ Cenário: Entrega de Encomendas

O Sistema Viário

Um país com N cidades conectadas por M estradas bidirecionais. Cada estrada possui um custo de pedágio P . O objetivo é viajar entre qualquer par de cidades com o menor custo possível.

O problema

O veículo quebrou em uma cidade fora da rota e foi consertado, a transportadora quer saber qual o caminho com menor custo

A Rota de Serviço

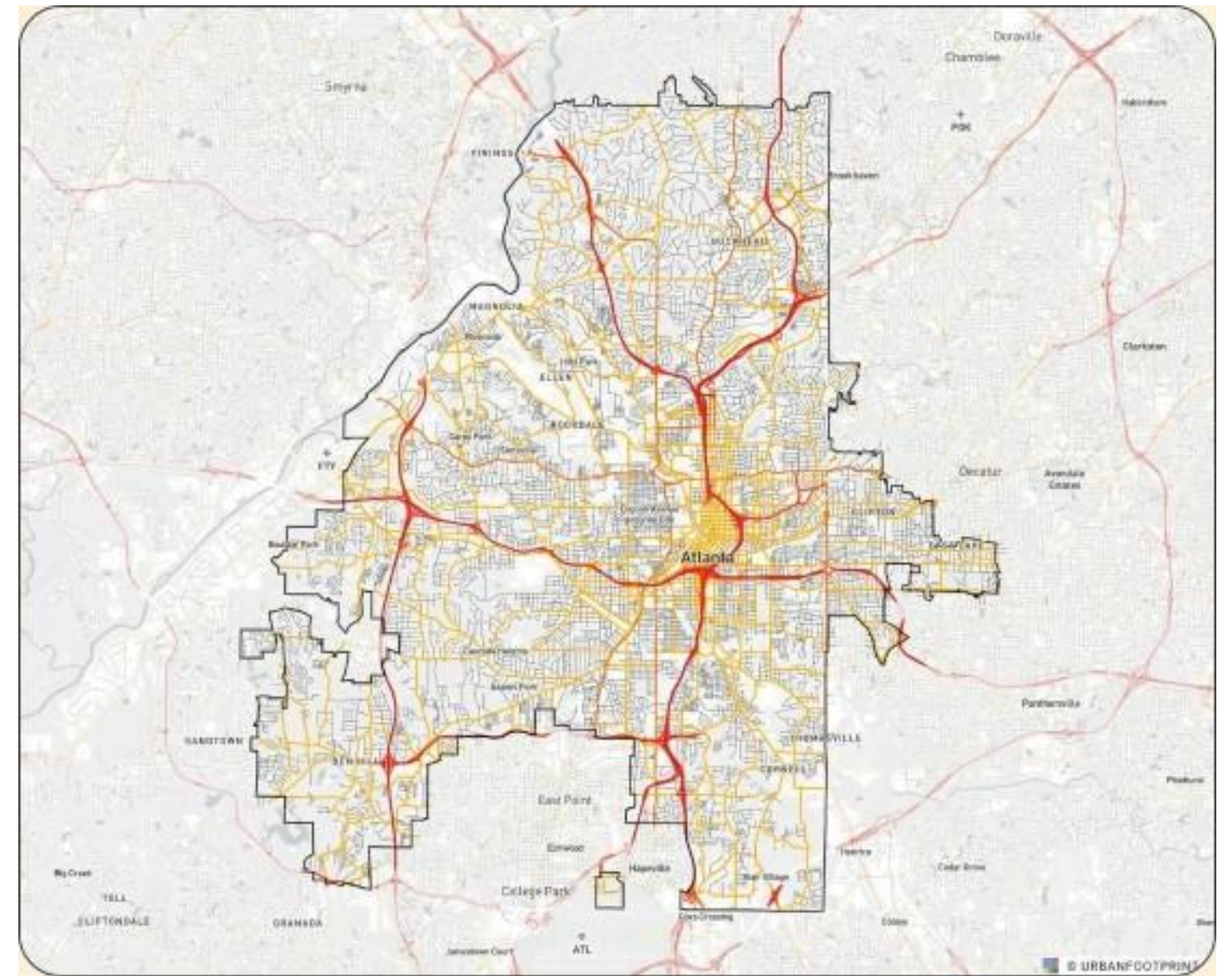
Para cada encomenda, define-se uma rota fixa de C cidades $(0, 1, \dots, C-1)$. Se um veículo quebrar e for reparado, ele deve voltar ao destino $C-1$ seguindo regras estritas se entrar na rota.

| Restrição Fundamental

A Regra de Ouro

Se em algum momento o veículo atingir uma das cidades que compõem sua rota de serviço (0 até C-1), ele é obrigado a seguir a sequência exata da rota (u a $u+1$) até o destino final.

Não é permitido sair da rota de serviço uma vez nela, nem "pular" cidades da sequência.



| Modelagem de Estados



Fora da Rota

Para cidades $u > C$: O veículo tem liberdade total. Pode escolher qualquer vizinho no grafo geral para tentar minimizar o custo.



Dentro da Rota

Para cidades $u < C$: O veículo perde a liberdade. O único caminho possível é para a cidade $u + 1$, com o custo do pedágio direto da rota.



O Destino

O objetivo final é sempre atingir a cidade $C - 1$, que é o ponto final da rota de serviço obrigatória.

Estratégia de Solução

```
dist = [float("inf")] * N
dist[K] = 0
pq = [(0, K)]

while pq:
    d_atual, u = heapq.heappop(pq)

    if d_atual > dist[u]:
        continue

    if u == C - 1:
        break

    if u >= C:
        for vizinho, peso in grafo[u]:
            if d_atual + peso < dist[vizinho]:
                dist[vizinho] = d_atual + peso
                heapq.heappush(pq, (dist[vizinho], vizinho))
    else:
        proximo = u + 1

        peso = pedagio_rota(u, proximo)
        if d_atual + peso < dist[proximo]:
            dist[proximo] = d_atual + peso
            heapq.heappush(pq, (dist[proximo], proximo))

print(dist[C - 1])
```

Dijkstra Modificado

Utilizamos o algoritmo de Dijkstra para encontrar o caminho mínimo (SSSP) da cidade de partida K até C-1.

A modificação reside no processo de Relaxamento: ao explorar vizinhos de um vértice, verificamos se ele pertence à rota obrigatória para filtrar as arestas permitidas.

| Lógica de Implementação

- Estrutura: O grafo é armazenado como lista de adjacência para eficiência no acesso aos vizinhos.
- @ Verificação de Rota: Mantemos um dicionário/mapa para consultar rapidamente o custo do pedágio entre (u, u+1) na rota sequencial.
- @ Fila de Prioridade: A biblioteca *heapq* do Python gerencia a Min-Priority Queue, garantindo que sempre processemos o menor custo acumulado primeiro.
- @ Poda Sugerida: Se o vértice atual for o destino (C-1), o algoritmo pode encerrar imediatamente para o caso de teste atual.

| Complexidade Algorítmica

$O(E)$

$\log V)$

Eficiência de Execução

A complexidade de tempo é dominada pela operação da Vila de Prioridade em cada aresta explorada.

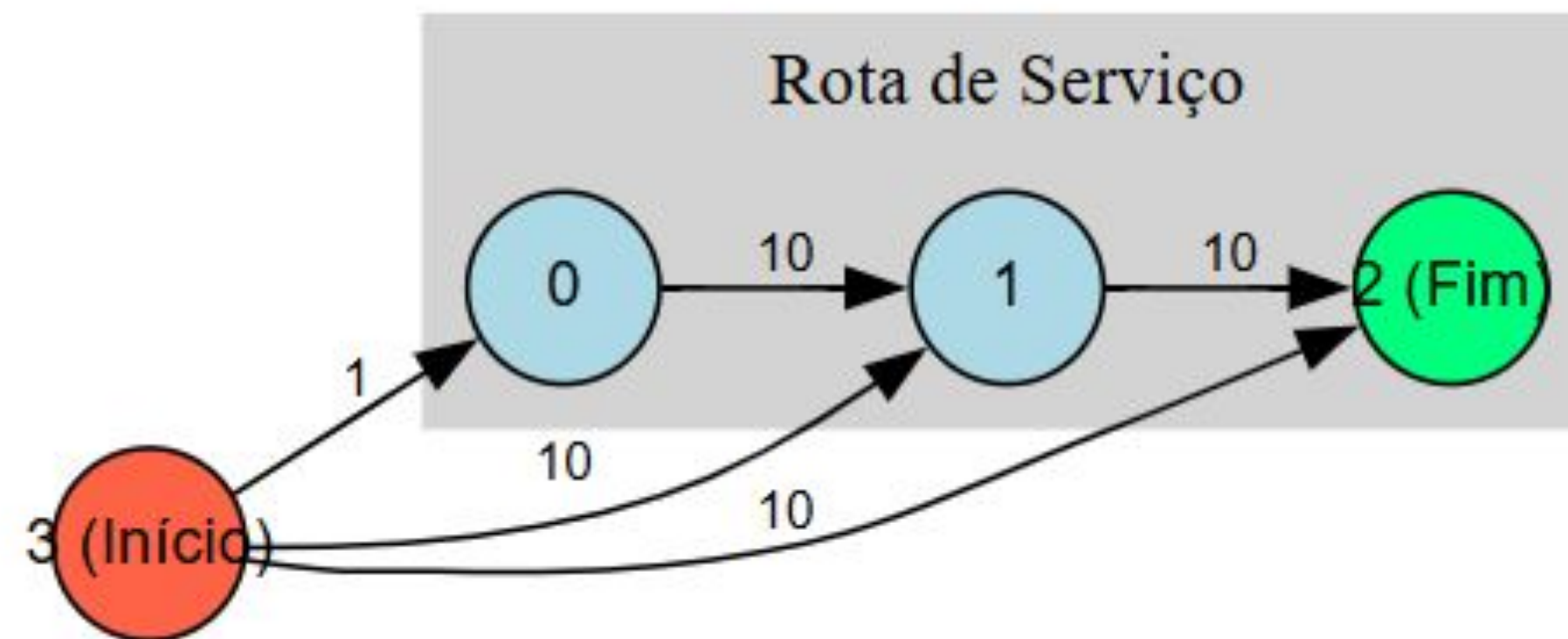
Para $N=250$ cidades, a solução é extremamente rápida, processando múltiplos casos de teste dentro do limite de tempo do UVa Online Judge.

Modelagem do Grafo

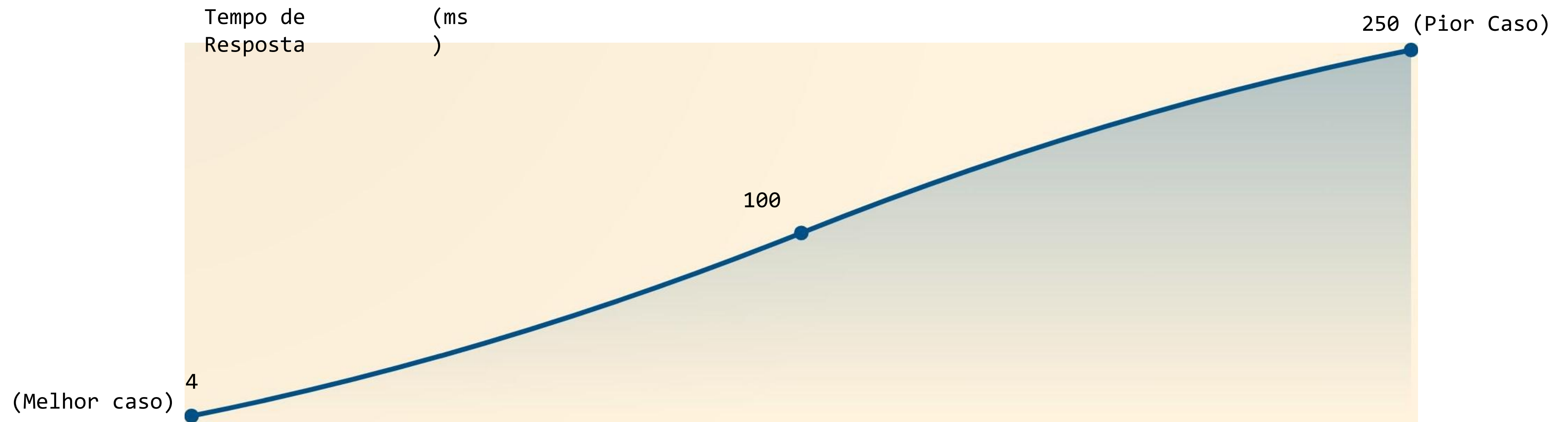
Representamos as conexões como um grafo pesado. No código, as arestas da rota sequencial são tratadas com prioridade lógica.

O relaxamento condicional garante que o veículo nunca "saia" da rota após entrar nela, refletindo fielmente a restrição do problema.

A Rota(1a entrada)



Desempenho com N Crescente



Comportamento log-linear estável conforme o tamanho do grafo aumenta.

Resultados de Exemplo

Caso de Teste	N, M, C, K	Resultado Esperado	Status
Exemplo 1	4, 6, 3, 3	10	SUCCESS
Exemplo 2	6, 7, 2, 5	6	VSUCCESS
Exemplo 3	5, 5, 2, 4	6	SUCCESS



Obrigado pela atenção!

Aaron Magno | Arthur Soares | Davi Dias